

FitFlow Manager — Perguntas e Respostas para Banca / Entrevista

Este arquivo contém perguntas prováveis sobre a arquitetura, funcionalidades e implementação do sistema, organizadas por nível de profundidade.

Sumário

- [Nível 1 — Perguntas Gerais](#)
 - [Sobre a Arquitetura](#)
 - [Sobre o Banco de Dados](#)
 - [Sobre o Funcionamento](#)
 - [Sobre Desafios e Decisões Técnicas](#)
 - [Nível 2 — Perguntas Aprofundadas \(Nível Código\)](#)
 - [Arquitetura — Detalhamento Técnico](#)
 - [Funcionalidades — Detalhamento Técnico](#)
 - [Implementação — Detalhamento de Código](#)
 - [Concorrência e Segurança](#)
-

Nível 1 — Perguntas Gerais

Sobre a Arquitetura

P: Por que vocês não usaram Spring Boot?

R: O foco do trabalho era JavaFX puro + JPA. Adicionar Spring Boot traria centenas de dependências, configuração de beans, injeção automática e complexidade desnecessária para um TCC. O servidor HTTP embutido do JDK (`com.sun.net.httpserver`) resolve a necessidade de uma API REST sem nenhuma dependência extra — pesa ~0 KB no JAR final.

P: Como o desktop e o mobile se comunicam?

R: O desktop JavaFX inicia um servidor HTTP na porta 8081 em uma thread separada. O mobile (SPA HTML/JS) faz requisições REST para esse servidor. O servidor usa os mesmos DAOs e EntityManager do desktop — ou seja, o banco é compartilhado. Toda operação no desktop ou mobile reflete no mesmo banco PostgreSQL.

Sobre o Banco de Dados

P: Por que `JOINED` em vez de `SINGLE_TABLE` na hierarquia `Usuario` ?

R: `SINGLE_TABLE` joga tudo numa tabela só com colunas `NULL` — desperdício e sem integridade. `JOINED` normaliza: cada subclasse (Admin, Instrutor, Aluno) tem sua própria tabela só com suas colunas específicas, e a chave primária é a mesma da tabela `usuario`. O banco garante a consistência via FK.

P: ON DELETE CASCADE no banco versus CascadeType no JPA — qual a diferença?

R: CascadeType do JPA opera **em memória**: quando você chama `em.persist()` ou `em.merge()` no pai, ele propaga a operação para os filhos **antes** de chegar no banco. ON DELETE CASCADE opera **no banco**: quando você deleta uma linha, o próprio PostgreSQL remove as linhas filhas. Usamos os dois: CascadeType.ALL para salvar em cascata (ex: ItemTreino → SerieTreino), e ON DELETE CASCADE no schema.sql como plano B para deleções via `em.remove()`.

P: Quantas tabelas o sistema tem?

R: 13 tabelas: usuario, admin, instrutor, aluno, avaliacao_fisica, treino, exercicio, item_treino, serie_treino, programacao_treino, sessao_treino, item_realizado, comentario_treino. Todas com ON DELETE CASCADE nas FKs.

Sobre o Funcionamento

P: Como o aluno acessa o sistema pelo celular?

R: O instrutor cadastra o aluno no desktop. O aluno abre o navegador do celular, acessa `http://<IP_DO_SERVIDOR>:8081/pages/login.html`, faz login com email e senha, e vê sua ficha de treino. O servidor HTTP está rodando na mesma máquina do desktop — qualquer dispositivo na mesma rede consegue acessar.

P: Como os GIFs são buscados e exibidos?

R: No desktop, o instrutor cadastra um exercício e clica em "Buscar GIF". O sistema consulta a API do GIPHY com o nome do exercício e grupo muscular, e salva a URL na coluna `url_midia`. Se a URL estiver vazia (exercício antigo ou sem GIF), o servidor mobile tenta buscar automaticamente na primeira vez que a ficha é carregada e salva no banco para não repetir a busca.

P: O que acontece quando o aluno finaliza um treino?

R: O app mobile envia um POST `/api/treino/finalizar` com os itens realizados (cada série, carga usada, se fez ou pulou). O servidor: (1) cria uma `SessaoTreino` vinculada à programação, (2) cria um `ItemRealizado` para cada exercício da sessão, (3) registra um `ComentarioTreino` com o feedback do aluno. O feed do aluno então exibe esse histórico.

Sobre Desafios e Decisões Técnicas

P: Qual foi a maior dificuldade técnica?

R: Integrar o servidor HTTP com o ciclo de vida do JavaFX. O servidor precisa iniciar depois que a UI está pronta, e ser encerrado quando a aplicação fecha. Além disso, como o servidor roda em threads separadas e o `EntityManager` do JPA não é thread-safe, cada handler precisa criar seu próprio `EntityManager` isolado — padrão `EntityManager-per-request`.

P: Como evitar que a UI congele durante operações de banco?

R: Toda operação JPA (especialmente consultas que podem ser lentas) roda em uma thread separada via `new Thread(() -> { ... }).start()`. Quando a operação termina, usamos `Platform.runLater(() -> { ... })` para voltar à thread do JavaFX e atualizar a UI com segurança. Sem isso, a tela congela até o banco responder.

P: O que você faria diferente hoje?

R: (1) Usaria um framework de build mais leve (Gradle), (2) adicionaria testes unitários desde o começo, (3) separaria melhor as responsabilidades dos controllers JavaFX (alguns estão fazendo trabalho demais — lógica de negócio misturada com UI), (4) usaria um pool de conexões JPA mais robusto (HikariCP), (5) hash nas senhas em vez de texto puro.

P: Como testar o sistema sem ter dois dispositivos?

R: O servidor mobile atende requisições HTTP comuns. Para testar no mesmo PC, abra `http://localhost:8081/pages/login.html` no navegador. O SPA funciona em qualquer navegador moderno — não precisa ser celular. Para testar como se fosse mobile, use o modo "inspetor responsivo" do navegador (F12 > toggle device toolbar).

P: Quais melhorias futuras estão planejadas?

R: (1) Hash + salt nas senhas, (2) notificações push para o mobile quando o instrutor atribuir nova ficha, (3) gráficos mais detalhados de progresso (evolução de carga por exercício ao longo do tempo), (4) modo offline para o mobile (cache com Service Worker), (5) exportação de relatórios em PDF, (6) testes automatizados com JUnit.

Nível 2 — Perguntas Aprofundadas (Nível Código)

Arquitetura — Detalhamento Técnico

P: Como o `PainelPrincipalController` gerencia o carregamento dinâmico de FXMLs no `StackPane` ? Como é o ciclo de vida de cada tela?

R: Cada botão da sidebar chama `trocarTela("Tela.fxml")` . O método instancia um `FXMLLoader` , carrega o FXML (que já instancia o controller via `fx:controller`), e chama `areaConteudo.getChildren().setAll(root)` . O `StackPane` substitui o filho anterior — o controller antigo perde referência e é coletado pelo GC. Cada controller tem um método `initialize()` chamado automaticamente pelo JavaFX após a injeção dos `@FXML` . Não há destroy — a limpeza depende do GC.

```
private void trocarTela(String fxml) {  
    try {  
        FXMLLoader loader = new FXMLLoader(getClass().getResource("/fxml/" + fxml));  
        Node tela = loader.load();  
        areaConteudo.getChildren().setAll(tela);  
    } catch (IOException e) { /* trata erro */ }  
}
```

P: O `HttpServer` do JDK cria uma thread por requisição ou usa pool? Como foi configurado?

R: O `HttpServer` por padrão cria uma thread nova para cada requisição, o que é ineficiente. Configuramos um pool compartilhado via

`servidorAtual.setExecutor(Executors.newCachedThreadPool())` (`ServidorMobile.java:60`). O `CachedThreadPool` reusa threads existentes e cria novas sob demanda, com tempo de vida de 60s de inatividade. Isso evita o overhead de criar/destruir threads para cada requisição.

P: O `EntityManager` do JPA não é thread-safe. Como o sistema lida com isso tendo o servidor HTTP multi-thread + JavaFX simultâneos?

R: Cada operação (handler HTTP ou ação JavaFX) cria seu próprio `EntityManager` via `JPAUtil.getEntityManager()` , usa em uma transação, e fecha ao final (`finally { em.close() }`). Isso segue o padrão "EntityManager-per-request". O `EntityManagerFactory` (criado uma única vez pelo `Persistence.createEntityManagerFactory()`) é thread-safe — ele é usado para criar `EntityManager` s isolados, cada um com seu próprio escopo de cache L1 e transação. Não há risco de condição de corrida entre handlers porque cada um manipula entidades detached/carregadas em seu próprio `EntityManager` .

```
EntityManager em = JPAUtil.getEntityManager();
try {
    em.getTransaction().begin();
    // operações...
    em.getTransaction().commit();
} catch (Exception e) {
    if (em.getTransaction().isActive()) em.getTransaction().rollback();
} finally {
    em.close();
}
```

P: O `com.sun.net.httpserver` é considerado API interna. Não há risco de remoção futura? Por que não usou Javalin ou Spark?

R: O pacote `com.sun.*` não faz parte da especificação oficial do Java SE, mas está presente desde o Java 6 (2006) — 18 anos de compatibilidade. A Oracle nunca deu sinais de removê-lo. Para um TCC, a escolha é pragmática: zero dependência extra, zero configuração, e o suficiente para servir uma SPA e uma API REST simples. Se o projeto fosse para produção, migraríamos para Javalin, Spark ou Spring Boot — todos usam a mesma abstração de `HttpExchange`. A migração seria trocar os handlers, não a lógica.

P: Como o `StaticFileHandler` serve os arquivos do SPA? Como lida com MIME types e path traversal?

R: O `StaticFileHandler` (`ServidorMobile.java:573-610`) resolve o caminho relativo da URL a partir de `src/main/resources/FitFlow app/`. Usa `getClass().getClassLoader().getResource()` para localizar o arquivo no classpath. O MIME type é inferido pela extensão do arquivo (`.html` → `text/html`, `.js` → `application/javascript`, `.css` → `text/css`). A proteção contra path traversal é garantida pelo próprio classloader: se alguém tentar `../../etc/passwd`, o `getResource()` retorna `null` porque o recurso não existe no classpath, e o handler retorna 404.

```
static class StaticFileHandler implements HttpHandler {
    @Override
    public void handle(HttpExchange ex) throws IOException {
        String path = ex.getRequestURI().getPath();
        if ("/".equals(path)) path = "/pages/app.html";
        InputStream in = getClass().getResourceAsStream("/FitFlow app" + path);
        if (in == null) { json(ex, 404, "{\"erro\":\"Arquivo não encontrado.\"}"); return;
    }

        String mime = path.endsWith(".html") ? "text/html" : path.endsWith(".js") ?
"application/javascript" : "text/css";
        ex.getResponseHeaders().set("Content-Type", mime + "; charset=UTF-8");
        ex.sendResponseHeaders(200, 0);
        in.transferTo(ex.getResponseBody());
        in.close();
    }
}
```

Funcionalidades — Detalhamento Técnico

P: Como o `FichasTreinoController.popularGrade()` monta a tabela editável de séries? O usuário pode editar células diretamente?

R: A tabela `tabelaFicha` usa colunas com `cellValueFactory` ligado a propriedades do objeto `ItemTreino`. A coluna de carga, por exemplo, usa `TextFieldTableCell` para edição inline: o usuário clica duas vezes na célula, digita o novo valor, e o `onEditCommit` atualiza o objeto em memória. As células são pré-preenchidas com valores padrão (ex: 4 séries, 10 reps, 50kg) quando o exercício é adicionado via `adicionarNaFicha()`.

```
colunaReps.setCellValueFactory(cellData ->
    cellData.getValue().getSeriesTreino().isEmpty() ?
        new SimpleStringProperty("-") :
        new
SimpleStringProperty(String.valueOf(cellData.getValue().getSeriesTreino().get(0).getRepeticoes()))
);
```

P: Como o gerador automático de treino (`clicouGeradorTreino`) distribui os exercícios? Qual a lógica?

R: O método `geradorListaPersonalizada()` no `FichasTreinoController` implementa uma heurística simples:

1. Pega todos os exercícios do catálogo e os agrupa por grupo muscular

2. Seleciona 1 exercício de cada grupo muscular prioritário (peito, costas, pernas, ombros)
3. Completa com exercícios de grupos secundários (bíceps, tríceps, abdômen) até atingir 8-10 itens
4. Para cada item, cria 4 séries de 10 reps com carga base (peso corporal para iniciantes)
5. Distribui intervalos de descanso: 60s para exercícios compostos, 45s para isoladores

A lógica é intencionalmente simples para ser didática. Um sistema real usaria regras mais sofisticadas (periodização, progressão de carga individualizada).

P: O modo "foco" do mobile — o timer continua contando se o usuário minimizar o app ou travar a tela?

R: Não. O JavaScript usa `setInterval()` que **não** executa em background no iOS/Android modernos. Quando o usuário minimiza o navegador, o sistema operacional pausa os timers JavaScript. Ao voltar, o timer continua de onde parou (o valor de `segundos` é preservado). Isso é uma limitação conhecida de SPAs sem Service Worker. Uma melhoria seria usar `document.visibilitychange` para pausar/retomar o timer explicitamente, ou usar `Date.now()` para calcular o tempo real decorrido.

P: Quando um instrutor edita uma ficha que o aluno já está usando, o que acontece? O aluno vê a versão antiga ou a nova?

R: O aluno vê a **versão nova** imediatamente na próxima vez que `carregarTreino()` for chamado (ao recarregar a aba Treino ou fazer pull-to-refresh). Isso porque a ficha é buscada do banco a cada requisição — não há cache no mobile. A `ProgramacaoTreino` aponta para um `Treino` específico via FK. Se o instrutor editar o `Treino` (adicionar/remover exercícios), o `BuscarFichaHandler` retorna os dados atualizados na próxima requisição GET `/api/ficha`. Se o instrutor criar uma **nova** `ProgramacaoTreino` para o mesmo aluno, o sistema retorna a mais recente (ordenada por `dataInicio DESC`).

P: Como o sistema calcula o "streak" (dias consecutivos de treino)? Qual a lógica?

R: No `TreinoDAO.buscarDadosDashboard()`, a lógica carrega todas as `SessaoTreino` do aluno ordenadas por data, e percorre a lista verificando se as datas são consecutivas:

```
int streak = 0;
LocalDate hoje = LocalDate.now();
for (SessaoTreino s : sessoes) {
    LocalDate data = s.getDataInicio().toLocalDate();
    if (hoje.minusDays(streak).equals(data)) streak++;
}
```

Isso considera que, se o aluno treinou hoje, conta dias consecutivos para trás. Se treinou ontem e hoje, streak=2. Se pulou um dia, zera.

P: Como a busca de GIF em lote funciona na tela de Exercícios? Quantas threads simultâneas são usadas?

R: Na tela `ExerciciosController`, o instrutor pode clicar "Buscar GIF" para um exercício específico (individual) ou "Buscar GIF para Todos sem GIF" (lote). Em lote, o controller:

1. Filtra exercícios com `urlMidia` vazia via stream
2. Itera sobre a lista e dispara `new Thread(() -> { ... }).start()` para cada exercício (usando `AtomicInteger sucesso` para contar e evitar race condition)
3. Na prática, com 30 exercícios, 30 threads disparam simultaneamente para a API do GIPHY
4. Cada thread, ao completar, chama `Platform.runLater()` para atualizar a tabela e salvar via `ExercicioDAO.salvar()`
5. O limite de taxa (rate limit) da GIPHY gratuita é gerenciado via `Thread.sleep(200)` entre disparos para não estourar o limite

```
List<Exercicio> semGif = todos.stream()
    .filter(e -> e.getUrlMidia() == null || e.getUrlMidia().isEmpty())
    .toList();
for (Exercicio e : semGif) {
    new Thread(() -> {
        String url = new GifSearchService().buscarMelhorGif(e.getNome(),
e.getGrupoMuscular());
        if (url != null) {
            e.setUrlMidia(url);
            new ExercicioDAO().salvar(e);
            sucesso.incrementAndGet();
        }
        Platform.runLater(() -> { /* atualizar UI */ });
    }).start();
}
```

Implementação — Detalhamento de Código

P: Como o `StringConverter` customizado exibe o nome do aluno no `ComboBox<Aluno>` em vez do `toString()` padrão?

R: O JavaFX `ComboBox` chama `toString()` do objeto para exibir no dropdown. Se não customizarmos, apareceria algo como `com.mycompany.academia.aluno.model.Aluno@1a2b3c`. Para mostrar o nome, configuramos um `StringConverter<Aluno>`:

```
comboBuscaAluno.setConverter(new StringConverter<Aluno>() {  
    @Override public String toString(Aluno a) {  
        return a == null ? "" : a.getNome() + " (ID:" + a.getId() + ")";  
    }  
    @Override public Aluno fromString(String s) { return null; /* só leitura */ }  
});
```

O mesmo padrão é usado para `ComboBox<ProgramacaoTreino>`, `ComboBox<Treino>` e `ComboBox<AvaliacaoFisica>` em diferentes telas. O método `fromString()` retorna `null` porque esses `ComboBoxes` são apenas para seleção — nunca para digitação.

P: Como o `TableUtils.autoFitColumns()` calcula a largura ideal? Por que não usar o `ColumnResizePolicy` do JavaFX?

R: O método percorre cada célula da coluna, mede o texto com `TextBoundsType` do JavaFX, e define a `prefWidth` como o maior valor entre o cabeçalho e os dados:

```
public static void autoFitColumns(Table<?, ?> tableView) {  
    for (TableColumn<?, ?> col : tableView.getColumns()) {  
        double maxWidth = 0;  
        for (int i = 0; i < tableView.getItems().size(); i++) {  
            Object value = col.getCellData(i);  
            Text text = new Text(value != null ? value.toString() : "");  
            Bounds bounds = text.getLayoutBounds();  
            maxWidth = Math.max(maxWidth, bounds.getWidth() + 20);  
        }  
        col.setPrefWidth(maxWidth);  
    }  
}
```

O `ColumnResizePolicy` padrão (`CONSTRAINED_RESIZE_POLICY`) distribui o espaço disponível igualmente entre colunas — o que não funciona bem quando uma coluna precisa de muito mais espaço que outra (ex: "Nome do Exercício" vs "Séries"). O `autoFitColumns()` ajusta cada coluna individualmente ao conteúdo, garantindo que textos longos não sejam cortados.

P: Como o `TrocarSenhaObrigatorioController` sabe que o usuário veio com senha "123456"? Onde a verificação acontece?

R: No `LoginController`, após `usuarioDAO.autenticar()` retornar o `Usuario`, verifica-se `"123456".equals(u.getSenha())`. Se verdadeiro, ao invés de abrir o `PainelPrincipal`, abre o `TrocarSenhaObrigatorio.fxml` passando o `Usuario` como parâmetro:

```
if ("123456".equals(u.getSenha())) {
    FXMLLoader loader = new
FXMLLoader(getClass().getResource("/fxml/TrocarSenhaObrigatorio.fxml"));
    Parent root = loader.load();
    TrocarSenhaObrigatorioController controller = loader.getController();
    controller.setUsuario(u);
    palco.setScene(new Scene(root));
} else {
    // abre PainelPrincipal normalmente
}
```

O `TrocarSenhaObrigatorioController` exibe a tela de troca, e ao salvar, chama `usuarioDAO.salvarSenha(u, novaSenha)`. Se o usuário fechar a janela sem trocar, ele não consegue acessar o sistema — não há como pular essa etapa.

P: Como o `AnaliseAlunoController` renderiza os gráficos de peso, IMC e carga? Usa JavaFX Charts (`LineChart`, `BarChart`) ou desenho manual?

R: Usa JavaFX Charts puros. Os gráficos são definidos no `AnaliseAluno.fxml` como `LineChart<Number, Number>` com IDs `graficoPeso`, `graficoImc`, `graficoCarga`. No controller, os dados são populados via `XYChart.Series`:

```
XYChart.Series<Number, Number> seriePeso = new XYChart.Series<>();
seriePeso.setName("Peso (kg)");
for (AvaliacaoFisica av : avaliacoes) {
    seriePeso.getData().add(new XYChart.Data<>(i++, av.getPeso()));
}
graficoPeso.getData().add(seriePeso);
```

A diferença do `graficoCarga` é que ele não usa `AvaliacaoFisica` — os dados vêm das `SessaoTreino` com os pesos levantados em cada exercício. A lógica de agregação (média de carga por sessão) está no `TreinoDAO.buscarSessoesPorAluno()`.

Padrões e Convenções de Código

P: Por que o código usa prefixo `o` em variáveis locais? Não é considerado antiquado?

R: O código segue as convenções ensinadas em aula, onde os exercícios práticos utilizam `o` para variáveis locais que armazenam objetos (`oAluno` , `oTreino` , `oDao`). Os parâmetros de métodos, entretanto, NÃO usam prefixo — seguindo o estilo real do professor visto nos exercícios originais (`prj_aula10_1` , `prj_aula5_2`).

A notação húngara tem a vantagem de tornar imediatamente visível o **tipo de escopo** de cada variável durante a leitura do código — essencial em métodos longos com múltiplas variáveis. Mantivemos o `o` para facilitar a correção e demonstrar domínio dos padrões apresentados em aula, mas alinhamos os parâmetros ao estilo autêntico do professor.

P: Por que os métodos dos DAOs têm nomes em português (`inserir` , `atualizar` , `excluir`) enquanto os getters/setters do JPA estão em inglês?

R: Os métodos de negócio (DAO e controller) seguem o vocabulário dos exercícios de aula, que usam português para operações de banco. Os getters/setters (`getNome()` , `setPeso()`) permanecem em inglês porque são gerados pela IDE e seguem a convenção JavaBeans, que o JPA exige para mapeamento. É uma separação clara: o que é regra de negócio fica em português; o que é infraestrutura do Java/JPA fica em inglês.

P: O que é `IEntidadeNomeada` e por que ela existe? Não seria mais simples usar uma classe abstrata?

R: `IEntidadeNomeada` é análoga à interface `IForma` dos exercícios de aula. Ela define o contrato `getNome()` para toda entidade que precisa ser exibida em ComboBoxes, labels e buscas. Optamos por interface (não classe abstrata) porque as entidades já estendem `Object` e são anotadas com `@Entity`. Uma interface permite que qualquer classe — inclusive as que já têm outra hierarquia — implemente o contrato sem conflito de herança.

P: Por que o DAO recebe a entidade como parâmetro do método (`dao.inserir(oAluno)`) em vez de receber no construtor (`new AlunoDAO(oAluno)`) como nos exercícios?

R: Nos exercícios de aula, o DAO recebe a entidade no construtor e a armazena em um campo, pois cada DAO trabalha com apenas uma entidade por vez. No FitFlow, preferimos passar a entidade como parâmetro do método porque isso permite reutilizar a mesma instância do DAO para múltiplas operações com diferentes entidades — sem precisar instanciar um novo DAO a cada chamada. Ambos os padrões estão corretos; a diferença é que o nosso é mais adequado para cenários com muitas operações encadeadas.

Concorrência e Segurança

P: As senhas estão em texto puro. Qual o custo de implementar BCrypt? O que precisaria mudar?

R: O custo é baixo. Precisaria: 1. Adicionar dependência `org.mindrot:jbcrypt:0.4` no `pom.xml` 2. Em `UsuarioDAO.salvar()`, trocar `u.setSenha(senha)` por `u.setSenha(BCrypt.hashpw(senha, BCrypt.gensalt()))` 3. Em `UsuarioDAO.autenticar()`, trocar `if (u.getSenha().equals(senha))` por `if (BCrypt.checkpw(senha, u.getSenha()))` 4. No `TrocarSenhaObrigatorioController`, usar `BCrypt.hashpw()` na nova senha 5. Executar um script único para hashear as senhas existentes no banco

O restante do sistema (sessões, tokens, FXMLs) não precisa mudar — a alteração fica encapsulada no DAO.

P: O SPA mobile armazena token e ID do aluno no `localStorage`. Qual o risco em dispositivo compartilhado?

R: O `localStorage` persiste os dados mesmo após fechar o navegador. Em um celular compartilhado, o próximo usuário que abrir o app estará logado como o anterior — a menos que o aluno clique em "Sair" (que chama `localStorage.clear()`). O risco é mitigado por: 1. Token sem expiração configurada (melhoria: adicionar expiração de 24h) 2. Qualquer pessoa com acesso físico ao celular pode ver os dados do treino 3. Solução ideal: usar `sessionStorage` (limpa ao fechar o navegador) em vez de `localStorage`, ou implementar biometria (Face ID / Touch ID) via WebAuthn

P: O que impede um aluno de chamar a API manualmente (ex: Postman) e obter dados de outros alunos?

R: Atualmente, a API valida o token de autenticação (`LoginHandler` gera um UUID e armazena em `sessoesAtivas`), mas não verifica se o `alunoId` da requisição corresponde ao token. Um aluno com token válido poderia teoricamente alterar o `alunoId` na URL para acessar dados de outro aluno. A mitigação seria: no `BuscarFichaHandler`, obter o `alunoId` do token mapeado (e não do parâmetro da URL) — ou verificar se `sessoesAtivas.get(token) == alunoId`. Essa é uma melhoria importante apontada na seção de segurança.

P: Dois instrutores podem editar o mesmo exercício ao mesmo tempo no desktop?

R: Tecnicamente, sim. O JavaFX não bloqueia o arquivo no banco. Se dois instrutores abrirem `FormExercicio.fxml` para o mesmo exercício, salvarem simultaneamente, o último `em.merge()`

sobrescreve o primeiro (lost update). O JPA não tem locking otimista/pessimista configurado. Para um TCC é aceitável; em produção, adicionaríamos `@Version` para optimistic locking:

```
@Version private int versao;
```

Se dois salvamentos concorrentes ocorrerem, o segundo lança `OptimisticLockException` e o usuário vê uma mensagem "Dados desatualizados, recarregue e tente novamente".